

Huginn

Assistant de santé psychologique et physique des colons

Dossier technique — Workshop 2026, Horizon 2080

Pilier HumanTech & HealthTech spatiales

Huginn est un prototype fictif pour un exercice de workshop. Le contenu médical (protocoles, prescriptions, inventaire) est illustratif, écrit pour la démonstration, et ne constitue ni un dispositif médical ni un avis médical réel.

1. Contexte et objectif

Dans un vaisseau interstellaire, l'équipage vit isolé pendant des mois, sans contact avec la Terre, sans médecin ni psychologue à bord. Fatigue, stress, troubles du sommeil et solitude peuvent dégrader la santé et la sécurité de la mission s'ils ne sont pas repérés tôt. Huginn (le corbeau de la pensée d'Odin, en écho à l'écusson Yggdrasil) analyse les constantes qu'un colon importe à la main depuis sa montre, décide de la conduite à tenir et le guide pas à pas, y compris pour des gestes de premiers secours.

Objectifs

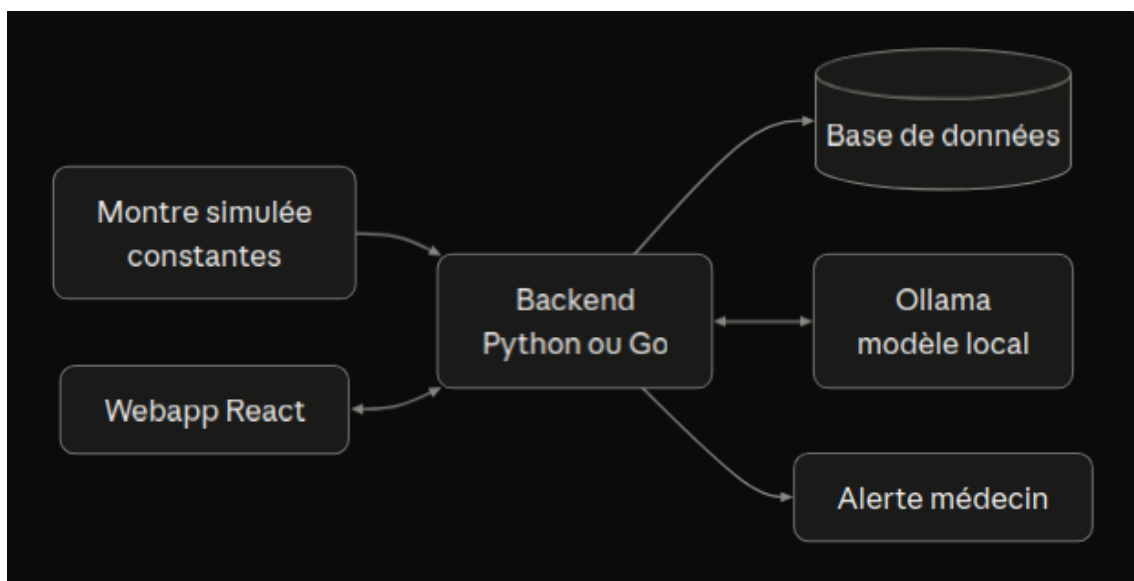
- Détecter tôt les signes de fatigue, de stress ou de mal-être à partir des constantes importées.
- Proposer des recommandations (repos, hydratation, exercice, respiration, activité sociale) et, si nécessaire, un protocole de premiers secours guidé avec prescription d'un médicament de l'inventaire de bord.
- Alerter l'équipage quand la situation dépasse ce que l'assistant peut gérer seul.
- Fonctionner sans aucune connexion extérieure.

2. Périmètre livré

Fonctionnalité	État
Connexion identifiant / mot de passe (comptes pré-crés)	Livré
Import manuel des constantes avec validation (type, plage)	Livré
État global (vert / orange / rouge / aucune donnée) et recommandations	Livré
Protocole guidé pas à pas + prescription + stock restant	Livré
Alerte à l'équipage, « Que faire ? » en lecture seule, résolution automatique	Livré
Graphiques 24 h / 7 jours, historique, filtre par type	Livré
Mode dégradé sans modèle IA	Livré et testé
Chat libre, réapprovisionnement, montre ESP32	Hors prototype

3. Architecture

L'ensemble tourne sur une seule machine, lancé par Docker Compose : un front React servi par nginx, un backend Python (FastAPI) avec base SQLite, et Ollama pour le modèle de langage local.



Couche	Technologie	Rôle
Front	React 19, Vite, Tailwind	3 pages responsive, connexion, formulaire d'import
Back	Python, FastAPI	API, authentification, validation, liaison IA, alertes, stock
IA	Ollama (llama3.2:3b par défaut)	Formulation de la recommandation, en local
Base	SQLite	Comptes, sessions, mesures, recommandations, alertes, suivis, stock
Protocoles	Fichiers JSON du dépôt	Contenu figé : étapes et prescription associée

Choix techniques tranchés

- **Python (FastAPI)** : écosystème simple, documentation d'API générée automatiquement.
- **SQLite** : aucune dépendance de service, fichier persisté sur un volume Docker.
- **llama3.2:3b** par défaut (rapide sur du matériel sans GPU), modèle réglable par variable d'environnement.
- **Pas de chat libre** : le guidage reste sous forme de cartes et d'étapes prédéfinies (cahier des charges §2).

4. Fonctionnement de l'analyse

À chaque import, le backend valide les valeurs (fréquence cardiaque 20–250 bpm, SpO2 0–100 %, température 30–42 °C, sommeil 0–24 h), les enregistre, puis calcule l'état par seuils. Un champ manquant ou une valeur hors plage bloque l'import (erreur 422 explicite).

Constante	Vert	Orange	Rouge
Fréquence cardiaque (bpm)	50 – 100	40 – 50 ou 100 – 120	< 40 ou > 120
SpO2 (%)	95 ou plus	90 – 95	< 90
Température (°C)	36,1 – 37,5	35,5 – 36,1 ou 37,5 – 38,5	< 35,5 ou > 38,5
Sommeil (h)	6 ou plus	4 – 6	< 4

L'état global est le pire niveau des quatre constantes. Le texte libre « symptômes » (facultatif) ne sert qu'à adapter le ton de l'IA : il n'entre jamais dans le calcul de l'état ni dans le choix du protocole.

Protocoles figés et prescription

Déclencheur (rouge)	Protocole	Prescription figée / alternative
SpO2	Détresse respiratoire / hypoxie	Bronchodilatateur inhalé / oxygène médical (masque)
Fréquence cardiaque	Fréquence cardiaque critique	Bêta-bloquant / anxiolytique léger
Température	Température corporelle critique	Vasopresseur d'urgence (absent du stock) / antipyrétique
Sommeil	Privation de sommeil critique	Aucune (repos guidé)

Quand plusieurs constantes sont critiques, un seul protocole est déclenché, par priorité : SpO2, fréquence cardiaque, température, sommeil. Le stock est commun à l'équipage et décrémenté de façon atomique. Si le médicament prévu n'est pas à bord (cas volontairement démontrable avec le vasopresseur), l'alternative prédéfinie est prescrite. Une alerte est créée pour l'équipage sans aucune constante ; elle se résout d'elle-même quand le colon atteint la dernière étape de son protocole.

Garde-fous de l'IA et mode dégradé

- L'IA ne rédige jamais d'instruction médicale ni de posologie : le contenu vient des fichiers figés.
- Toute réponse du modèle contenant une posologie ou un nom de médicament est rejetée et remplacée par le moteur de règles.
- Si Ollama ne répond pas dans le délai (8 s par défaut) ou échoue, le moteur de règles prend le relais : recommandations de bien-être citant une constante et déclenchement du bon protocole, sans dépendre du modèle.
- Un import produit plusieurs cartes (repos, respiration, hydratation...). Le moteur de règles choisit les thèmes selon les seuils ; chaque carte est rédigée par un appel distinct à l'IA, avec une consigne courte (1 à 2 phrases simples, tutoiement, une constante citée). Si un appel échoue, seule cette carte retombe sur le texte fixe du moteur de règles.

5. Sécurité et confidentialité

- Le colon est toujours déduit du jeton de session, jamais d'un paramètre client : un colon ne voit jamais les données d'un autre.
- Mots de passe hachés (PBKDF2-SHA256, 260 000 itérations) ; jetons opaques expirant après 12 h.
- Blocage après 5 échecs de connexion par minute et par identifiant ; calcul de hachage factice pour ne pas révéler les comptes existants par le temps de réponse.
- L'équipage voit qu'un colon est en danger, jamais le détail de ses constantes.
- CORS limité aux origines du front ; paramètre de période validé ; requêtes SQL via l'ORM (pas de concaténation).
- Aucun appel à Internet : le modèle, les protocoles et la base sont locaux.

6. Tests et validation

Huit tests automatisés (pytest) rejouent le scénario de démo en mode dégradé : connexion refusée, état « aucune donnée », vert, orange, rouge avec protocole et stock décrémenté, isolation entre colons, alerte visible de l'équipage sans constantes, résolution automatique, repli sur le médicament alternatif, validation des entrées, blocage du login, expiration de session, garde-fou IA. Le parcours complet a aussi été joué dans un navigateur (Chromium, format mobile) : connexion, import, protocole guidé, bandeau et « Que faire ? » côté équipage.

7. Scénario de démonstration

- Erik se connecte : état « aucune donnée », invitation à importer une première mesure.
- Constantes normales : l'état passe au vert.
- Fréquence cardiaque en hausse et nuit courte : orange, recommandations de respiration, repos et hydratation.
- Constantes de crise (par exemple SpO2 à 88 %) : rouge, protocole d'hypoxie guidé pas à pas, prescription et stock restant, alerte à l'équipage.
- Nyota, autre colon connecté, voit l'alerte, clique sur « Que faire ? » et consulte le même protocole en lecture seule ; quand Erik termine le protocole, l'alerte disparaît.
- On coupe le modèle IA : le mode dégradé continue de fonctionner.

8. Limites et perspectives (V1)

- Vérification à faire sur le matériel du campus : modèle réel (3B contre 7B) et temps de réponse cible inférieur à 10 s.

- Chat libre avec l'assistant, gestion des interactions médicamenteuses, allergies et historique de traitement.
- Réapprovisionnement et gestion de pénurie ; connexion d'une montre ESP32.
- Notifications en temps réel (WebSocket) à la place du sondage toutes les 8 secondes.

Huginn est un prototype fictif pour un exercice de workshop. Le contenu médical (protocoles, prescriptions, inventaire) est illustratif, écrit pour la démonstration, et ne constitue ni un dispositif médical ni un avis médical réel.